

1. 前置作業

硬體:

CPU: Intel® Core™ i7-11800H Processor 2.3 GHz

GPU: NVIDIA® GeForce RTX™ 3060 Laptop GPU

RAM:16GB DDR4 SO-DIMM

軟體:

Windows 10 Home

開發軟體:

Spyder IPython 7.29.0

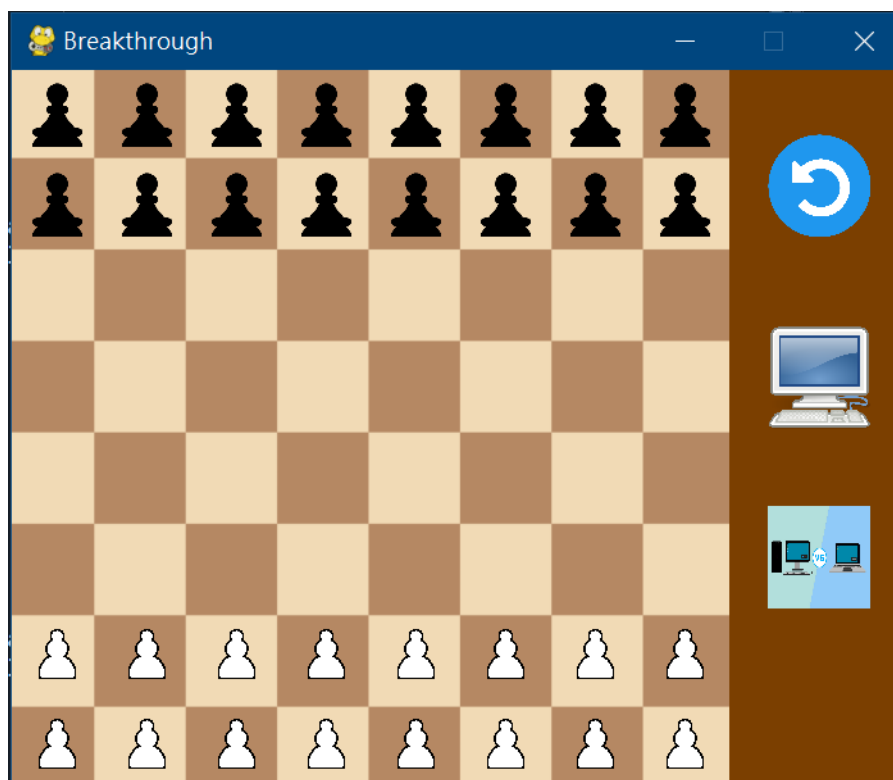
程式語言:

Python 3.9.7

連絡電話: 0909182511

2. 使用說明

(1) 啟動 game.py



(2) 決定遊戲模式

按下以下圖示所在位置，決定接下來的遊戲狀況

I. 與電腦對戰

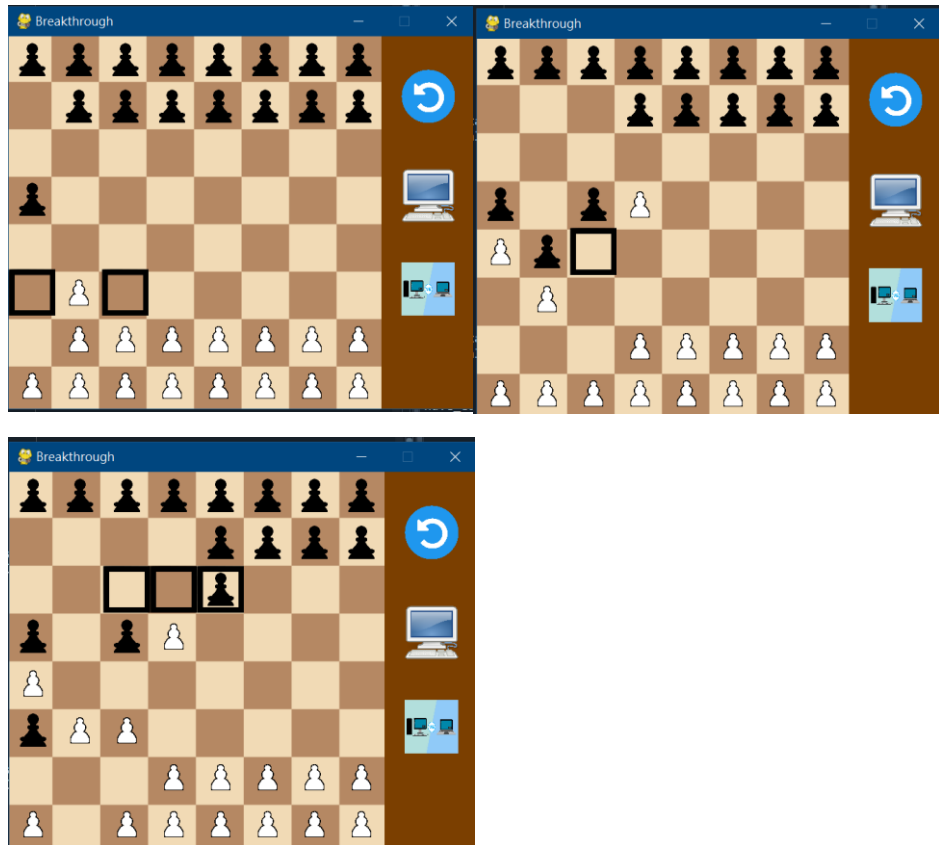


開始與電腦對弈，黑棋(電腦)會先開始，完成移動後，會以黑框顯示電腦移動棋子的前後位置。



輪到玩家選擇要移動的白棋，玩家選則玩棋子後，會以黑框顯示合法移動的位置，包括吃掉黑棋時要遵照的規則。





玩家移動完白棋後，會自動輪到電腦選擇，不需再按圖示

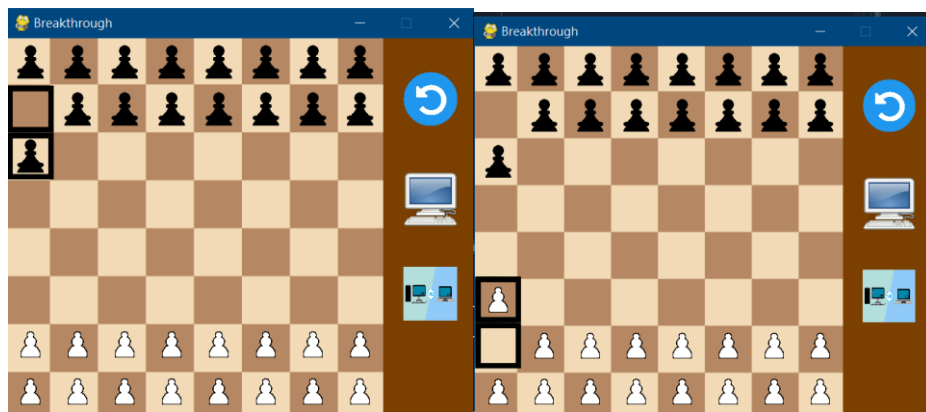
當一方滿足勝利條件後，顯示遊戲結束。



II. 自動對戰(電腦對戰電腦)

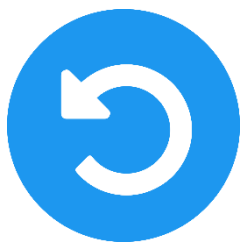


電腦會輪流移動黑棋和白棋，移動前後的棋子都會輪流以黑框標示出來。



遊戲持續到一方勝利，並顯示結束圖示。

(3) 重新開始遊戲



遊戲盤面回復到最初狀態，並讓玩家再次選擇要進行的模式。

(4) 退出遊戲視窗

關閉遊戲視窗，程式便會直接終止

3. 程式說明

使用的 Class:

BreakthroughGame

記錄介面的顯示畫面跟互動狀況，以及各種遊戲模式下雙方對戰的機制及運作流程，並判斷是否滿足勝利條件。

State

記錄盤面狀況，黑棋與白棋各自的位置，並在雙方選擇移動路徑時判斷移合法的移動位置。

玩家選擇時，顯示合法位置並交給介面控制 **BreakthroughGame** 顯示出來。

電腦移動時，則交給 **AlphaBetaAgent** 判斷在合法位置中要選擇的位置。

AlphaBetaAgent

為電腦選擇移動位置時的演算法，採用 alpha-beta 篩選的方式。

期望值計算方式為盤面我方留存的棋子數加上所在位子的盤面垂直座標數值，分別記錄雙方期望值並相減。

選擇時，選擇對方期望值最小的情況，自己期望值越大的局面

Action

記錄這一盤面移動的棋子座標和方向以及移動的選手是哪一方，並在介面以黑框顯示電腦移動得過程

4. 遇到的問題

這是我第一次使用介面將程式資料顯示出來，一開始對於介面互動的突發狀況並沒有處理好，導致出現一些錯誤情況。

(1) 滑鼠按下位置不在預定位置中，導致程式判斷出現異常

```
Traceback (most recent call last):  
  File "D:\alex1\Documents\OneDrive\文件\師大\三下\人工智慧\hw3\game.py", line 626, in  
<module>  
    main()  
  File "D:\alex1\Documents\OneDrive\文件\師大\三下\人工智慧\hw3\game.py", line 624, in main  
    game.run()  
  File "D:\alex1\Documents\OneDrive\文件\師大\三下\人工智慧\hw3\game.py", line 156, in run  
    if self.boardmatrix[coor_x][coor_y] == self.turn:  
IndexError: list index out of range
```

解決方式：

在判斷滑鼠位置時都要預留錯誤位置，並在未點擊正確位置時都維持程式當前運行狀態

(2) 選擇棋子後，想要另外選擇其他棋子卻被判定選擇結束，造成棋子消失的異常情況



解決方法：

與第一個狀況不同的是，這次點擊位置錯誤但不合理，造成程
是判斷錯誤並移除棋子。這時不應該維持當前程是狀態，而是
要在判斷不合理後取消這次狀態並返回前一狀態

5. 參考資料

<https://github.com/gibson31/BreakthroughGame/blob/master/breakthroughgame.py>

參考其介面設計以及判斷方法和流程

<http://portal.lib.ntnu.edu.tw:8080/server/api/core/bitstreams/c380d9e6-0a6c-47fe-8f37-b4412bf05978/content>

參考關於 breakthrough 的遊戲規則

<https://fu-sheng-wang.blogspot.com/2017/02/ai-16-alpha-beta-pruning.html>

參考關於 alpha-beta 的演算法方式

<http://search.taai.org.tw/paper/2017/0/%E9%9B%BB%E8%85%A6Breakthrough%E6%A3%8B%E7%A8%8B%E5%BC%8FLittle%20Giant%E7%9A%84%E8%A8%AD%E8%A8%88%E8%88%87%E9%96%8B%E7%99%BC.pdf>

參考關於 breakthrough 期望值的計算